



A novel blockchain-enabled zero-trust-based authentication scheme in power IoT environments

Siwei Li¹ · Hui Zhang¹ · Hui Shi¹ · Maode Ma² · Cong Wang³

Accepted: 21 May 2024 / Published online: 3 June 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

The distributed authentication of a large number of resource-constrained power terminal devices (PTDs) is crucial for ensuring the security of the power IoT communication. However, existing solutions are inadequate in terms of security (such as vulnerability to single point failures or insider attacks) and communication costs, which do not meet the requirements of the power IoT environment. Therefore, we propose a zero-trust-based mutual authentication scheme in cloud-edge-end collaboration power IoT environment based on blockchain technology. The proposed solution involves deploying a distributed multi-point zero-trust engine around dense PTDs to collect real-time identity information. Through the maintenance of an alliance blockchain, the edge server securely shares and traces identity information, preventing tampering and effectively blocking threats related to lost terminals during authentication. Additionally, a lightweight ECC key management scheme ensures the security of authentication information. The proposed scheme effectively defends against common attacks such as replay attacks and identity forgery attacks through informal security analysis, while its security is evaluated using the Canetti and Krawczyk (CK) adversary models. Performance evaluation results demonstrate that the proposed scheme achieves effectiveness without the compromising required security attributes, which obtains up to 58% improvement 58% improvement in computation delay and a 56.9% improvement in communication costs over previous state-of-the-art methods.

Keywords Zero trust · Blockchain · Authentication · Power IoT

✉ Cong Wang
wangcongjcdd@tust.edu.cn

¹ Beijing FibrLink Communications Co. Ltd., Beijing 100071, China

² KINDI Center for Computing Research, College of Engineering, Qatar University, Doha, Qatar

³ Tianjin University of Science and Technology, Tianjin, China

1 Introduction

The Power IoT is an industrial-grade application of the IoT in the power industry, aimed at enhancing the perception, interaction, and adjustment capabilities of the power grid. It ensures secure, stable, continuous, and efficient operation across all aspects of "power generation, transmission, transformation, distribution and utilization." Being a critical infrastructure related to the national economy and people's livelihoods, the power IoT has always been a prime target for cyber attacks [1]. The power terminal devices (PTD) cover the terminal equipment in the fields of power transmission and transformation, customer side, and supply chain [2].

The growing use of advanced technologies like "big cloud, smart chain, and edge computing" in the power industry has led to a significant rise in the number and types of PTDs connected to the power IoT. This increased connectivity poses a serious challenge to network security defenses that rely on boundary isolation. Research indicates that many smart PTDs have inherent security risks, including vulnerabilities in their firmware with manufacturer-installed backdoors. The security of these intelligent PTDs is crucial for the overall safety and stability of the power IoT. If PTDs are compromised, they may lose their ability to control operations normally, allowing attackers to exploit them for intrusion or identity theft. Compromised devices can also serve as springboards for internal threats, bypassing border network security defenses. Once attackers gain entry through a springboard attack, they can manipulate data and send malicious control commands, posing a significant threat.

Following the principle of "never trust, always authenticate," all devices, users and traffic in the network are required to be authenticated and authorized, zero-trust has been recognized as an effective means to resist internal threats [3]. However, current research primarily focuses on the centralized deployment of zero-trust engines, which poses challenges in meeting the authentication requirements for large-scale PTD access in power IoT scenarios. Moreover, if the zero-trust engine is deployed too far from the PTDs, it may result in delayed authentication times that hinder the widespread adoption of zero-trust in delay-sensitive power IoT.

1.1 Motivation

To ensure the security of PTD access to the power Internet of things and protect data transmission from illegal intrusion, and improve the active defense capability of the power Internet of things, PTD security authentication is required. However, existing schemes still encounter safety or efficiency issues. For instance, the schemes in [4–7] fail to provide anonymity guarantees, while the schemes in [6, 8, 9] are susceptible to de-synchronization attacks. Additionally, the schemes in [9, 10] are vulnerable to KCI attacks. Solutions based on blockchain technology proposed in [11] do not adequately protect user privacy. The schemes discussed in [11, 12] are prone to side channel attacks. The schemes presented in [11, 13, 14] exhibit high computational costs and poor communication efficiency.

Additionally, there are a large number of devices in the power Internet of things, so a decentralized security system should be adopted to build a light-weight key management system. Blockchain is a technology that allows the creation of a distributed digital ledger of transactions, and its ledger is shared with all nodes in the network. With the characteristics of subject reciprocity, openness and transparency, secure communication, difficult tampering and multi-party consensus, blockchain has been applied to more and more fields [15, 16]. Given the authentication needs arising from extensive PTDs access in the power IoT, it is crucial to broaden the application scope of zero trust from centralized systems to decentralized edge environments.

1.2 Contributions

Through strategic deployment of zero-trust engines (ZTE) encircling densely concentrated PTDs, we can proficiently alleviate risks linked with compromised PTDs, thereby establishing an edge-centric zero-trust model. The main contributions of this paper are as follows:

- 1) The zero-trust network internal monitoring and authentication functions are deployed at the edge of the power IoT, in close proximity to the PTDs. Through distributed multi-point deployment of ZTEs, it effectively mitigates single point failure and prevents denial-of-service attacks.
- 2) An ECC-based mutual authentication scheme for the cloud-edge-end collaboration power IoT environment based on blockchain technology is proposed, called EMAS. EMAS enables bidirectional authentication between the edge server and ZTE while ensuring efficiency.
- 3) EMAS avoids certificate operations and optimizes the content stored on the blockchain. In addition, the distributed storage nature of blockchain can solve the single point of failure problem of edge server.
- 4) Canetti and Krawczyk (CK) attack model [17] is used to analyze EMAS, and it is found that the EMAS is secure under the random oracle model. Through informal security analysis, EMAS scheme can resist some typical attacks, such as DDoS attacks and impersonation attacks.
- 5) For performance evaluation, EMAS obtains up to 58% improvement in computation delay and a 56.9% improvement in communication costs over previous state-of-the-art methods.

1.3 Organization

The remaining content of this article is organized as follows:

In Sect. 2, the related works are presented, followed by the definition of the ECC public key cryptosystem, system model and threat model in Sect. 3. In Sect. 4, the details of EMAS are introduced. In Sect. 5, the security of EMAS is analyzed. Section 6 evaluates and compares EMAS's key performance indicators. Finally, Sect. 7 summarizes the full text and the direction of future work.

2 Related works

The significance of zero trust in the industry has become increasingly prominent as cybersecurity boundaries blur. Following the Google Aurora incident, Google initiated a research program focused on zero trust and adopted the BeyondCorp [18] architecture as an exemplary model, making it the first successful implementation to truly embody the concept of zero trust. This architecture defines the security relationship among users, devices, and applications by establishing a chain of trust through continuous authentication, enabling authorized users to access protected services. Saleem et al. proposed the adoption of a zero-trust security model in the SaaS cloud computing environment for large-scale data analysis of multimedia forensic security information processing [19]. Chen et al. focus on the security risks faced by the current power mobile Internet and design a zero-trust-based security protection framework from two aspects: identity identification and access control [20]. Sultana et al. proposed a defense system for vulnerabilities in medical health data based on the zero-trust model [21]. Faced with the bottleneck of centralized deployment of zero-trust due to massive power terminal access, Feng Jingyu et al. introduced an edge zero-trust model that deploys zero-trust engines through distributed multi-points around dense power terminals [22]. However, this model has limitations in analyzing internal traffic characteristics. Additionally, integrating blockchain and power IoT also presents various challenges, including architectural-level considerations and data privacy.

Extensive efforts have been made to ensure the security of terminal access to the power IoT and data transmission from the threat of illegal intrusion to improve the active defense capability of the power IoT, which includes encryption methods based on Rivest, Shamir, and Adleman (RSA) encryption [4, 5] and elliptic curve encryption (ECC) [6–10, 23–25]. Uludag et al. [4] proposed an identity authentication scheme using RSA encryption for the power IoT environment, which can realize two-way authentication and resist replay attacks, but is vulnerable to anonymous attacks. Philips et al. [5] proposed an enhanced RSA encryption algorithm for power IoT, which improves the strength and speed of key encryption by changing the selection of prime numbers. Srinivas et al. [6] designed a certificate-free authentication scheme based on ECC for power IoT environment, which allows the dynamic addition of user terminals, resists replay attacks and analog attacks, and has high efficiency and cost advantages. However, Srinivas' solution has some security flaws, such as not ensuring user anonymity and being vulnerable to impersonation attacks. The scheme proposed by Xiang et al. [7] cannot also guarantee user anonymity and has low efficiency, which cannot fully meet the requirements of power IoT.

In order to ensure the anonymity of users, Deng et al. [10] proposed an identity-based anonymous aggregate signature scheme for the power IoT, which reduces the computing cost by avoiding certificate management. Chang et al. [23] proposed a privacy protection scheme for anonymous data sharing in the power IoT, which can ensure the confidentiality and integrity of data. Chaudhry et al. [8] proposed a power IoT authentication scheme based on ECC that can resist a

variety of attacks, including replay attacks and man-in-the-middle attacks. However, Chaudhry's scheme was not resistant to desynchronization attacks, and the large number of ECC point multiplications resulted in high computation costs. Hu et al. [24] proposed an identity authentication and key protocol scheme for advanced metering infrastructure of the power IoT based on ECC, which can resist anonymous attacks and transient secret disclosure attacks, and at the same time has good forward confidentiality. However, the session keys in Hu's scheme are not independently generated and, once exposed, are vulnerable to attack. Abbasinezhad-Moodet et al. [25] proposed an anonymous key establishment protocol called AKEP that does not require server participation. The AKEP scheme uses a large number of point multiplications, resulting in a high latency. Qi et al. [9] proposed a smart grid key authentication scheme based on ECC, called TPAK, which does not require linear pairing, thus reducing the computing time. However, TPAK cannot resist KCI attacks and is vulnerable to denial-of-service attacks.

Compared with the above authentication protocols, the scheme in [11–14, 26] makes full use of the strong privacy protection, distributed storage, and tamper-proof features of blockchain technology, thereby efficiently ensuring security. Blockchain technology also helps solve some problems with high computing costs and low ability to resist cyber attacks [27]. The solution in [6, 14, 28, 29] uses the decentralized technology of blockchain to solve the single point of failure problem in the power IoT. Javed et al. [26] proposed a blockchain-based certificate authentication scheme, which uses Hyper-elliptic curve cryptography (HECC) to define the blockchain as the certificate issuer and realize the authentication without trusting a third party. However, Javed's solution has some shortcomings in terms of user privacy protection. Wang et al. [11] proposed a blockchain-based edge computing anonymous authentication scheme named BAAK. However, BAAK is vulnerable to side channel attacks and impersonation attacks, and anonymity is not fully guaranteed. Naseer et al. [25] propose a scheme that eliminates the need for trusted third parties and central entities. However, according to Naseer's scheme, all the generated key information is stored on the local device, which carries the risk of insufficient information security and vulnerability to side-channel attacks. The scheme in [13, 14] has some problems in terms of efficiency. Therefore, the research and improvement in these schemes is still an important research field for the improvement in the security of the power IoT, which requires continuous exploration and innovation.

Numerous studies have explored data management techniques in the power IoT networks for the analysis and processing of data generated by various smart devices [28, 29]. However, the data generated by smart devices is vulnerable to attack threats, as these technologies are often provided over open channels. If a malicious attacker breaches a smart device, they can disrupt the balance between power demand and response, tampering with the smart device's data. Therefore, the power IoT needs a privacy-protected authentication scheme to cope with various attacks. Existing solutions are inadequate in terms of security [6–8, 10, 13, 24, 25] and often adopt a centralized approach to monitor and manage devices, which is less efficient [13, 14]. At present, the secure access system of the power IoT still uses the traditional digital certificate authentication method of the power communication network, but with the

development of the power IoT, many power IoT terminal devices are connected to the intelligent IoT system, and the existing digital certificate authentication method is expensive to calculate and is not suitable for the power IoT. Therefore, an authentication scheme that supports decentralized device management based on zero trust and protects privacy has become an indispensable security requirement. The security features between the proposed EMAS scheme and the other related schemes are compared in Table 1.

3 Preliminary and system model

In this section, we will first introduce the relevant knowledge and definition of the ECC followed by the introduction of the system model and threat model.

3.1 ECC

Elliptic curve cryptography (ECC) is a public-key cryptography algorithm based on the principles of elliptic curve mathematics. Compared to traditional encryption algorithms such as RSA, ECC offers higher performance and smaller key sizes with the same level of security, which makes it more suitable in resource-constrained environments such as the power IoT.

Table 1 Comparison of security features among related schemes

	[4]	[6]	[8]	[9]	[10]	[11]	[25]	Ours
F1	Y	Y	Y	Y	Y	Y	Y	Y
F2	N	N	Y	Y	Y	N	Y	Y
F3	N	Y	Y	Y	Y	Y	Y	Y
F4	Y	Y	Y	Y	Y	Y	Y	Y
F5	Y	Y	Y	Y	Y	N	Y	Y
F6	Y	N	N	N	Y	Y	Y	Y
F7	Y	N	Y	N	N	Y	Y	Y
F8	Y	Y	Y	N	Y	Y	Y	Y
F9	Y	Y	Y	N	Y	Y	Y	Y
F10	N	N	Y	Y	N	N	Y	Y
F11	Y	N	–	–	N	Y	–	Y
F12	Y	N	Y	Y	N	Y	Y	Y
F13	Y	Y	N	Y	N	N	N	Y

F1: Resistance to replay attacks, F2: Resistance to impersonation attacks, F3: Resistance to MIMT attacks, F4: Resistance to unknown key-share attacks, F5: Resistance to modification attacks, F6: Resistance to time synchronization attack, F7: Resistance to ephemeral secret leakage, F8: Known key security, F9: Perfect forward secrecy, F10: Smart Meter Anonymity, F11: Smart Meter Untraceability, F12: Formal security proof, F13: Low computation delay

The basic principle of ECC is to use the addition and multiplication of points on the elliptic curve to achieve the generation of public and private keys, data encryption and decryption, as well as digital signatures and other cryptographic operations. Its security is based on the discrete logarithm problem on an elliptic curve, i.e., finding a point P on the elliptic curve such that $kP = Q$, where k is the private key and Q is the public key. In the elliptic curve discrete logarithm problem, it is computationally infeasible to deduce the private key from the public key, thus ensuring the security of ECC.

The group of elliptic curves $E_p(a, b)$ is defined by the non-singular elliptic curve equation E over a finite field of prime numbers F_p , where p is prime as shown in Eq. 1.

$$E_p(a, b) : y^2 = x^3 + ax + b \pmod{p}, a, b \in F_p, \Delta = 4a^3 + 27b^2 \pmod{p} \neq 0 \quad (1)$$

Lemma 1 (Elliptic Curve Discrete Logarithm Problem (ECDLP)) *Given the sum of fixed points $G \in E_p(a, b)$ and $P = kG \in E_p(a, b)$, the discrete logarithm problem is very hard to calculate $k \in \mathbb{Z}_q^*$.*

Lemma 2 (Elliptic Curve Diffie–Hellman Problem (ECDHP)). *The security of the ECDHP key exchange system is based on the security of ECDLP. Given $G, xG, yG \in E_p(a, b)$, the calculation $xyG \in E_p(a, b)$ is very difficult.*

3.2 System model

As a crucial technology of the power IoT, edge computing is essentially a distributed computing architecture. It involves decentralizing cloud computing and storage capabilities to the network's edge, enabling local, close-range, and distributed deployment of applications, services, and content. Conversely, local distributed protection has emerged as a vital concept for ensuring the stable operation of the power IoT. Considering this aspect, it is feasible to deploy internal monitoring and authentication functions provided by zero trust to each edge region divided based on the "generation, transmission, transformation, distribution and use" aspects of the power system. This allows for distributed deployment and identity management of zero trust engines around PTDs. The power IoT scenario can establish an edge zero trust model based on an "end," "edge," and "cloud" system architecture. The architecture of power IoT consists of four parts: home local area network (HAN), local area network (LAN), neighborhood local area network (NAN), wide area network (WAN), and blockchain network (BCN) [6], as shown in Fig. 1.

HAN is the first layer of the network system, which mainly includes an integrated sensing array, various sensors, inspection robots, etc. [30]. The installation of a zero-trust client (ZTC) on each networked PTDs enables real-time monitoring of its network activity and resource access requests. The ZTC's risk-sensing mechanism promptly blocks any suspicious behavior or abnormal traffic. However, for resource access requests from the PTD, it is generally challenging for the ZTC to determine

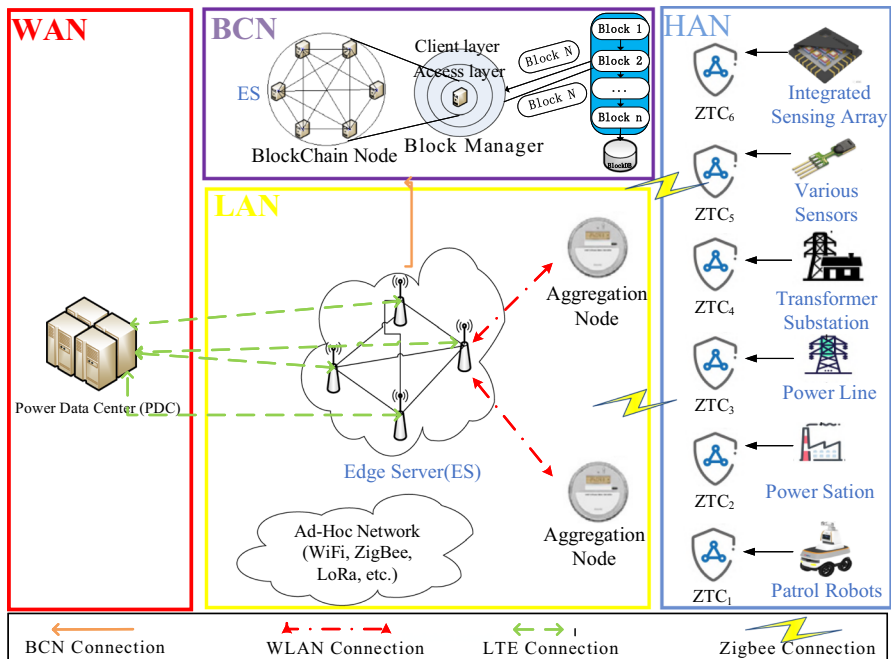


Fig. 1 The architecture of the power IoT

suspicious situations accurately. Therefore, these requests need to be submitted to the zero-trust engine for authentication and judgment.

As the second layer of the system, the LAN acts as a "bridge" between the HAN and WAN, consisting primarily of sink nodes and access nodes, sending data collected from the edge PTD to the edge server (ES) via WiFi, and then to the power data center (PDC) via LTE technology. Multiple edge servers are built as BCN network to upload critical information to the blockchain. Each ES joins the BCN network to guarantee the blockchain's normal work.

As the third layer of the system, the WAN is typically deployed by power and energy providers. Multiple LANs can be connected to the WAN via a wireless mesh network. PDCs are essential for facilitating processes such as electricity transmission, distribution, and sales. By deploying the zero-trust monitoring and authentication function at the network edge, PDC only needs to verify and issue authorization credentials before providing corresponding services based on the type of resource access request. This approach significantly enhances authentication efficiency while ensuring internal security within the power IoT.

3.3 Attack model

In this paper, a well-known and generally accepted confrontation model, the CK threat model [17], is adopted, which is a game between Defender $\mathcal{D}$ and Opponent $\mathcal{A}$.

The game is modeled by a probabilistic polynomial-time Turing machine (P.P.T.). Let Π_{Ω}^{ℓ} represent an instance ℓ of participant, where Ω represents a ZTC_i or anES_j. In the threat model, $\mathcal{A}$ can perform a series of actions. In order to reduce the difficulty of $\mathcal{A}$, $\mathcal{D}$ must respond according to the following rules.

- **Hash(m)**: $\mathcal{A}$ performs a hash query on message x . $\mathcal{D}$ checks whether message m has been asked before. If yes, the corresponding hash value of message m is returned. Otherwise, a random number r is returned and the record (m, r) is stored in the list.
- **Send (P, m)**: Send queries are defined based on modification attacks, replay attacks, and simulated attacks. By sending the query, $\mathcal{A}$ sends message m to participant $\mathcal{D}$ and will receive a response message by $\mathcal{D}$.
- **Execute(ZTC_pES_j)**: Execute query is defined on the basis of a passive eavesdropping attack which returns a copy of the information passed between the participants in the system. $\mathcal{A}$ may get the session key by collecting the information passed by both parties.
- **Test(ZTC_pES_j)**: $\mathcal{A}$ can submit this query only once. When P receives a Test query, an unbiased coin b is flipped. If $b = 1$, the actual session key is returned. Otherwise, a random value with the same length is returned to $\mathcal{A}$.
- **ESReveal (D)**: $\mathcal{A}$ is allowed to obtain $\mathcal{D}$'s ephemeral key by an ESReveal query.
- **Corrupt (D)**: $\mathcal{A}$ obtains the long-term key of $\mathcal{D}$ by Corrupt query.
- **Semantic Security-Mutual Authenticated-Security (MA-Security)**: $\mathcal{A}$ repeatedly performs an aforementioned query. If $\mathcal{A}$ randomly outputs a random number $b' \in \{0, 1\}$ such that $b' = b$, $\mathcal{A}$ wins the game. Let $Succ$ denote the success event and $\Pr[Succ]$ represents the probability of winning the game of MA-Security. Therefore, the advantage of $\mathcal{A}$ in breaking semantic security can be expressed as $\text{adv}_{\mathcal{A}}^{\text{MA}} = |2\Pr[Succ] - 1|$, and if there exists $\varepsilon > 0$ such that $\text{adv}_{\mathcal{A}}^{\text{MA}} < \varepsilon$, it can be defined as secure under the CK adversary model.

4 The proposed EMAS scheme

In this section, the proposed EMAS scheme is described in detail. Based on the system model, an ECC-based mutual authentication scheme for the cloud-edge-end collaboration power IoT environment based on blockchain technology is proposed. EMAS is divided into the following four stages, which are system initialization, registration, initial authentication and key agreement, and continuous authentication.

Figure 2 shows the system model for power IoT, which is composed of user, mobile APP, power terminal device (PTD), gateway, and edge server (ES). PTDs are some lightweight devices collecting and monitoring data through various heterogeneous IoT configurations, such as Smart light bulbs, smart meters (SM), smart monitoring, and sensors. The installation of a zero-trust client (ZTC) in each networked PTDs group enables real-time monitoring of its network activity and resource access requests. Gateways and ESs are both capable devices, which are devices with sufficient computing, storage, and communication resources to

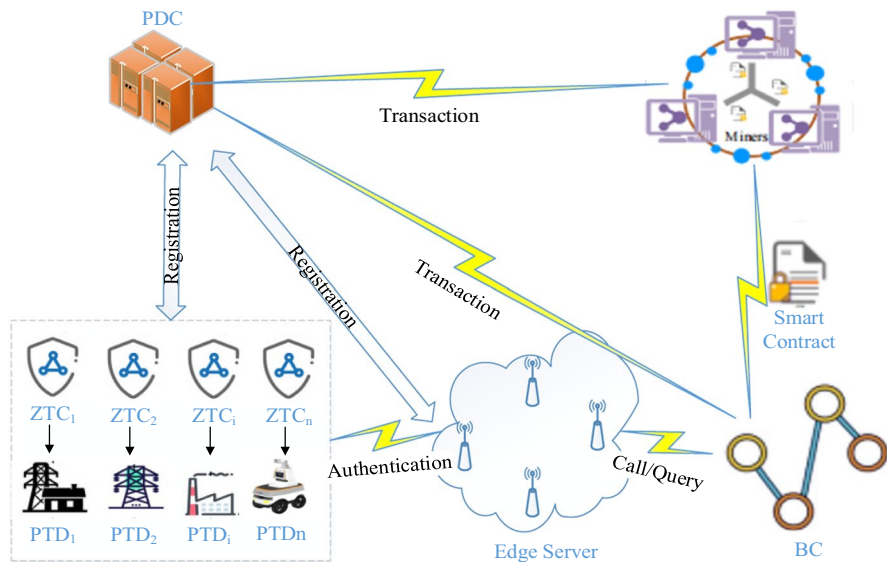


Fig. 2 The system model of proposed EMAS scheme

act as full nodes of the blockchain network and perform operations such as transaction creation, verification, and query. The gateway stores the data generated by the device layer and provides it to users as needed. ESs act as edge clouds and can handle a large number of operations. The power data center (PDC) only needs to verify and issue authorization credentials based on the type of resource access request. This paper focuses on the mutual authentication between ZTC and ES.

The identity authentication process of ZTCs consists of two phases: initial authentication and continuous authentication (Table 2). During continuous authentication, ZTC and ES undergo mutual authentication at regular intervals, denoted ast_c . Additionally, to enhance security, they must perform initial authentication periodically within a designated interval t_i . Similarly, to mitigate security risks stemming from potential key leakage, PDC selects a new node and regenerates public parameters and keys for both entities after a certain time interval t_k . It

Table 2 Notations

Notations	Description
<i>PTD</i>	Power terminal devices
<i>ZTC</i>	Zero-trust client
<i>ES</i>	Edge server
<i>PDC</i>	Power data center
$H_i(i=0, 1, 2, 3)$	Secure hash function
R_i, x, y	Random numbers
ET_i	Expiry timestamp

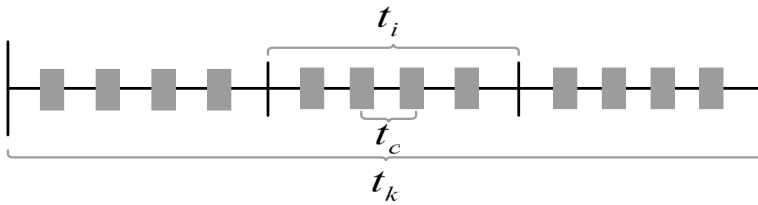


Fig. 3 Time interval of the authentication process

is worth noting that ($t_c < t_i < t_k$), as illustrated in Fig. 3. Furthermore, if both entities are trusted, for efficiency purposes, we may extend the time intervals t_k and t_i .

4.1 System initialization phase

In the system initialization phase, *PDC* initializes all the required system parameters for the proposed EMAS scheme as follows.

4.1.1 Basic initialization

- On the non-singular elliptic curve, *PDC* chooses a large prime q , a cyclic additive group G , a multiplicative group $G \times G \rightarrow G_T$, a key generator P as the generator of G and G_T . *PDC* calculates a bilinear mapping $e: G \times G \rightarrow G_T$, $g = e(P, P)$.
- *PDC* selects a random number $s \in \mathbb{Z}_q^*$ as its private key, and then calculates its own public key $P_{pub} = sP$.
- *PDC* selects four secure hash functions $H_i : \{0, 1\}^* \times G \rightarrow \mathbb{Z}_q^*$, $i = 0, 1, 2, 3$.
- *PDC* issues a set of system parameters $(G, G_T, q, P, P_{pub}, H_0, H_1, H_2, H_3)$.

4.1.2 Blockchain initialization

PDC creates a genesis file with configuration parameters, which is used to establish the blockchain. *PDC* then selects several trusted partners and launches the blockchain according to a specific consensus mechanism (e.g., Practical Byzantine Fault Tolerance in FISCO BCOS). For simplicity, *PDC* can be added directly to an existing blockchain system (such as FISCO BCOS).

4.1.3 Smart contract deployment

In our proposed EMAS, the smart contract is employed to manage the key materials table. By leveraging this underlying smart contract, the system can offer conditionally anonymous authentication for communicators and facilitate efficient revocation without encountering asynchronous issues. The corresponding smart contracts are briefly illustrated in Algorithms 1 to 4.

Algorithm 1 Initialization_EMAS

```

contract EMAS{
    address owner;
    % Define the structure of components in EMAS.
    struct EMAS {
        Byte32 PID;
        byte32 PUB;
        DateTime expiryTime;}
    EMAS[] public EMAS;
    % Constructor, automatically invoked when the smart contract is deployed.
    constructor EMAS() {
        owner = msg.sender;
        len = 0;
        return 1; }
}

```

Algorithm 2 Update_EMAS

```

function update_EMAS(oldPID, PID, PUB, expiryTime){
    % Invoked by the PDC to update EMAS.
    if owner != msg.sender then
        return 0;
    else {
        % if there exists a tuple of <PID, -> in EMAS, update it; otherwise, add a new tuple for it.
        if Exist(EMAS[i].PID == oldPID) then {
            EMAS[i].PID = PID;
            EMAS[i].PUB = PUB;
            EMAS[i].expiryTime = expiryTime;
            return 1;}
        else {
            len++;
            EMAS[len].PID = PID;
            EMAS[len].PUB = PUB;
            EMAS[len].expiryTime = expiryTime;
            return 1; }}
}

```

Algorithm 3 query_EMAS

```

function query_EMAS(PID){
    % Invoked by the ES to retrieve specific public key materials.
    if Exist(EMAS[i].PID == PID) then
        return EMAS;
    else
        return 0;
}

```

Algorithm 4 revoke_EMAS

```

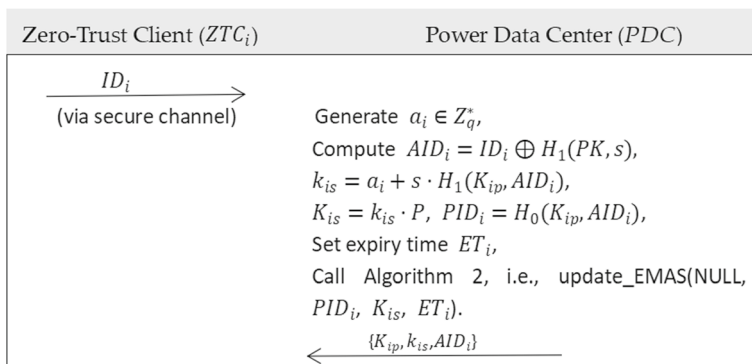
function revoke_EMAS(PID){
    % Invoked by the PDC to revoke a ZTC.
    if owner  $\neq$  msg.sender then
        return 0;
    else {
        % If there exists the tuple of <PID,> in EMAS(), delete it.
        if Exist(EMAS[i].PID == PID) then {
            Release(EMAS[i]);
            for ; i<len; i++
                EMAS[i] = EMAS[i+1];
            Len --;
            return 1;}
        else
            return 0; }
    }

```

4.2 Registration phase**4.2.1 ZTC_i registration**

ZTC_i Registration process is shown in Fig. 4. ZTC_i sends its ID_i to PDC for registration via a secure channel. After receiving the registration request, PDC first checks whether it has been registered. If yes, PDC will interrupt the registration request. Otherwise, PDC calculates the key material in the following way.

- PDC first calculates the anonymous identity $AID_i = ID_i \oplus H_1(PK, s)$.
- PDC generates a random number $a_i \in Z_q^*$, as well as calculates $K_{ip} = a_i \cdot P$, $k_{is} = a_i + s \cdot H_1(K_{ip}, AID_i)$, and $K_{is} = k_{is} \cdot P$.

**Fig. 4** PTD_i registration process

- Then PDC computes $PID_i = H_0(K_{ip}, AID_i)$, sets an expiry time ET_i for ZTC_i .
- PDC uploads $\langle PID_i, ET_i \rangle$ to the blockchain by calling Algorithm 2, i.e., $update_EMAS(NULL, PID_i, K_{is}, ET_i)$.
- PDC sends $\langle K_{ip}, k_{is}, AID_i \rangle$ to ZTC_i .

After receiving the information, ZTC_i first verifies whether the equation $k_{is} \cdot P == K_{ip} + H_1(K_{ip}, AID_i) \cdot PK$ is valid. If yes, ZTC_i sets (k_{is}, K_{is}) as the private and public key pair.

4.2.2 ES_j registration

ES_j Registration process is shown in Fig. 5. ES_j sends its ID_j to PDC for registration via a secure channel. After receiving the registration request, PDC first checks whether it has been registered. If yes, PDC will interrupt the registration request. Otherwise, PDC calculates the key material in the following way.

- PDC generates a random number $a_j \in Z_q^*$, as well as calculates $K_{jp} = a_j \cdot P$, $k_{js} = a_j + s \cdot H_1(K_{jp}, ID_j)$, and $K_{js} = k_{js} \cdot P$.
- Then PDC computes $PID_j = H_0(K_{jp}, ID_j)$, and sets an expiry time ET_j for PID_j .
- PDC uploads $\langle PID_j, ET_j \rangle$ to the blockchain by calling Algorithm 2, i.e., $update_EMAS(NULL, PID_j, K_{js}, ET_j)$.
- PDC sends $\langle K_{jp}, k_{js} \rangle$ to ES_j .

After receiving the information, ES_j first verifies whether the equation $k_{js} \cdot P == K_{jp} + H_1(K_{jp}, ID_j) \cdot PK$ is valid. If yes, ES_j set (k_{js}, K_{js}) as the private and public key pair.

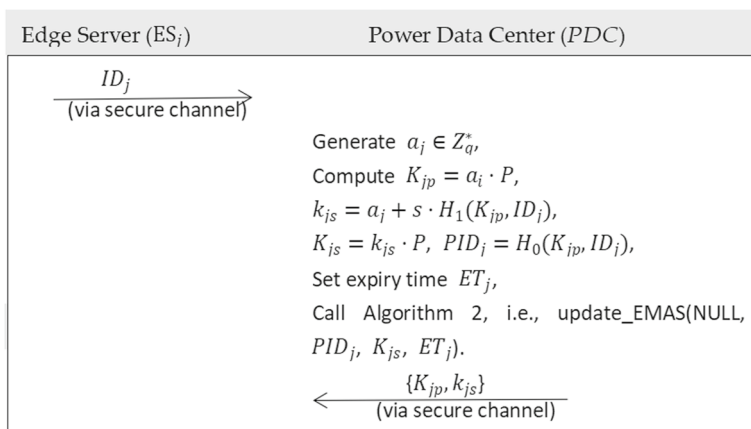


Fig. 5 ES_j registration process

4.3 Initial authentication and key agreement phase

The authentication and key agreement phase between ZTC_i and ES_j is shown in Fig. 6., which only includes two messages.

4.3.1 $ZTC_i \rightarrow ES_j : M_1 = (X_i, Q_i, K_{ip}, AID_i)$

ZTC_i selects the random number $x_i \in Z_q^*$ and calculates $X_i = x_i \cdot P$.

ZTC_i calculates $Q_i = x_i + k_{is} \cdot H_3(K_{is}, AID_i, X_i)$.

ZTC_i sends $(X_i, Q_i, K_{ip}, AID_i)$ to ES_j .

4.3.2 $ES_j \rightarrow ZTC_i : M_2 = (X_j, Q_j, K_{jp}, AID_j)$

- After receiving the message from ZTC_i , ES_j first will calculate $PID'_i = H_4(K_{ip}, AID_i)$.

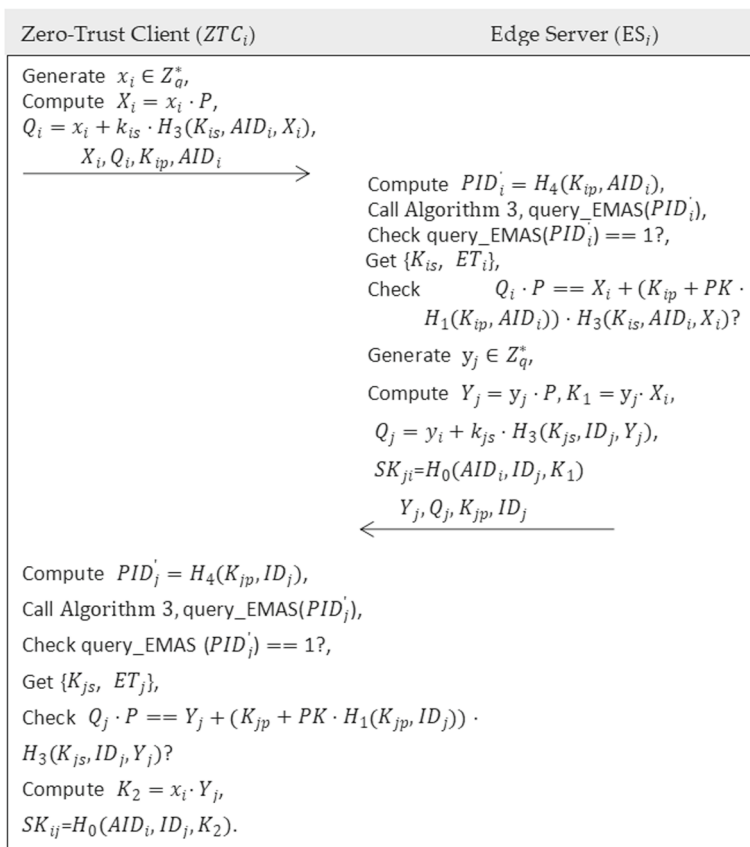


Fig. 6 The authentication process

- Then ES_j calls Algorithm 3, i.e., $\text{query_EMAS}(PID'_i)$. If the return result of Algorithm 3 is 0, ES_j will break the request, which means the requested ZTC_i is not registration for the PDC . Otherwise, ES_j can obtain the public key K_{is} and its expiration time ET_i of ZTC_i .
- ES_j checks the returned ET_i whether expired, if not, ES_j will verify ZTC_i by verifying that $Q_i \cdot P == X_i + (K_{ip} + PK \cdot H_1(K_{ip}, AID_i)) \cdot H_3(K_{is}, AID_i, X_i)$ is true as shown in Eq. 2, then. Otherwise, ES_j will break the request.

$$\begin{aligned} Q_i \cdot P &= (x_i + k_{is} \cdot H_3(K_{is}, AID_i, X_i)) \cdot P = x_i \cdot P + (a_i + s \cdot H_1(K_{ip}, AID_i)) \cdot \\ &H_3(K_{is}, AID_i, X_i) \cdot P = x_i \cdot P + (K_{ip} + s \cdot P \cdot H_1(K_{ip}, AID_i)) \cdot H_3(K_{is}, AID_i, X_i) \quad (2) \\ H_3(K_{is}, AID_i, X_i) &= X_i + (K_{ip} + PK \cdot H_1(K_{ip}, AID_i)) \cdot H_3(K_{is}, AID_i, X_i) \end{aligned}$$

- ES_j selects random number $y_j \in Z_q^*$ and calculates $Y_j = y_j \cdot P$ and $K_1 = y_j \cdot X_i$.
- ES_j calculates $Q_j = y_i + k_{js} \cdot H_3(K_{js}, ID_j, Y_j)$.
- ES_j sets the session key $SK_{ji} = H_0(AID_i, ID_j, K_1)$.
- ES_j sends (Y_j, Q_j, K_{jp}, ID_j) to ZTC_i .

After receiving the message M_2 from ES_j , ZTC_i first will calculate $PID'_j = H_4(K_{jp}, ID_j)$. Then ZTC_i calls Algorithm 3, i.e., $\text{query_EMAS}(PID'_j)$. If the return result of Algorithm 3 is 0, ZTC_i will terminal the session, which means the responded ES_j not registration for the PDC . Otherwise, ZTC_i can obtain the public key K_{js} and its expire time ET_j of ES_j .

- ZTC_i checks the returned ET_j whether expired, if not, ZTC_i will call Algorithm 4, i.e., $\text{authenticate_EMAS}(Y_j, Q_j, K_{jp}, ID_j, K_{js})$.
- ZTC_i verifies ET_j successfully if the equation $Q_j \cdot P == Y_j + (K_{jp} + PK \cdot H_1(K_{jp}, ID_j)) \cdot H_3(K_{js}, ID_j, Y_j)$ is valid. Otherwise, ZTC_i will terminal the session.
- ZTC_i sets the session key $SK_{ij} = H_0(AID_i, ID_j, K_2)$.

Because $K_2 = x_i \cdot Y_j = x_i \cdot y_j \cdot P = y_j \cdot x_i \cdot P = y_j \cdot X_i = K_1$, then $SK_{ij} = SK_{ji}$ can be concluded. The session key SK_{ij} or SK_{ji} can be used to encrypt the messages transmitted between ES_j and ZTC_i .

4.4 Continuous authentication

After initial authentication, both ZTC_i and ES_j possess the same session key SK_{ij} . In the continuous authentication phase, ZTC_i and ES_j authenticate each other relying on these results, as shown in Fig. 7.

1. $ZTC_i \rightarrow ES_j : \{CM_i, CA_i\}$

- ZTC_i generates a random number R_i .
- ZTC_i computes $CM_i = R_i \oplus H_3(PID_i, SK_{ij})$ and $CA_i = \text{HMAC}_{SK_{ij}}(PID_i, R_i, CM_i)$.

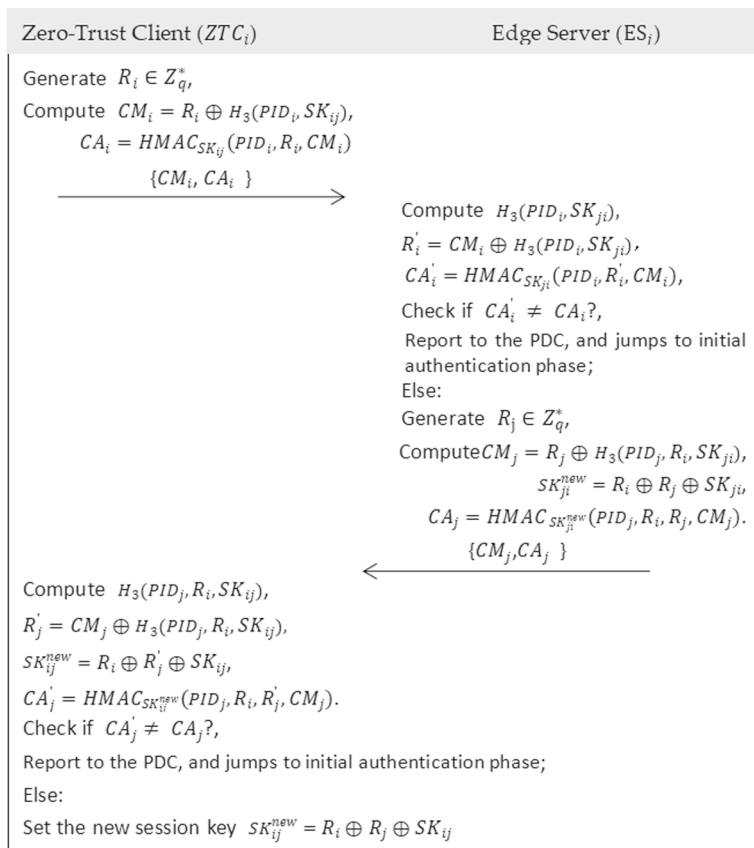


Fig. 7 The continuous authentication process

- ZTC_i sends CM_i and CA_i to the ES_j .
2. $ES_j \rightarrow ZTC_i : \{CM_j, CA_j\}$
 - ES_j computes $H_3(PID_i, SK_{ji})$, $R'_i = CM_i \oplus H_3(PID_i, SK_{ji})$ and $CA'_i = HMAC_{SK_{ji}}(PID_i, R'_i, CM_i)$.
 - ES_j checks whether CA'_i equals to CA_i . If not, ES_j will reports to the PDC, and jumps to the initial authentication phase.
 - ES_j generates a random number R_j .
 - ES_j computes $CM_j = R_j \oplus H_3(PID_j, R_i, SK_{ji})$, and $SK_{ji}^{new} = R_i \oplus R_j \oplus SK_{ji}$.
 - ES_j computes $CA_j = HMAC_{SK_{ji}^{new}}(PID_j, R_i, R_j, CM_j)$.
 - ES_j sends CM_j and CA_j to the ZTC_i .
 3. Once ZTC_i receives the message from the ES_j , ZTC_i does the following operations:
 - ZTC_i computes $H_3(PID_j, R_i, SK_{ij})$ and $R'_j = CM_j \oplus H_3(PID_j, R_i, SK_{ij})$.

- ZTC_i computes $SK_{ij}^{new} = R_i \oplus R'_j \oplus SK_{ij}$, and $CA'_j = HMAC_{SK_{ij}^{new}}(PID_j, R_i, R'_j, CM_j)$.
- ZTC_i checks whether CA'_j equals to CA_j . If not, ZTC_i will reports to the PDC, and jumps to the initial authentication phase.

Until now, ZTC_i and ES_j get mutual authentication and the key agreement with the new session key $SK_{ij}^{new} = SK_{ji}^{new} = R_i \oplus R_j \oplus SK_{ij}$.

4.5 Update and revocation

4.5.1 Update

There are also two cases that ZTC_i should update the key materials. In the first case, when it reaches the expiry time ET_i and the ZTC_i is still valid in the smart system, the PDC will generate new private key and corresponding public key with the same steps as those described in the registration phase. Note that the K_{is} is also updated to prevent traceability. In the second case, if ZTC_i 's private key is compromised, ZTC_i has to ask for the key update and then the PDC helps to update the key materials. When there is an update event for ZTC_i , the PDC invokes the smart contract interface `update_EMAS(oldPID, PID, Kis, ETi)`.

4.5.2 Revocation

There are also two cases that ZTC_i 's (or ES_j 's) private key and public key will get revoked. In the first case, if the PDC discovers suspicious behavior/activities from ZTC_i , the PDC sends a revocation transaction, and deletes the entry (PID_i, K_{is}, ET_i) from the smart contract. In the second case, if ZTC_i wants to leave the system, it sends a revocation request to PDC, and the PDC will revoke ZTC_i 's keys accordingly. When there is a revocation event for ZTC_i , the PDC invokes the smart contract interface `revoke_EMAS(PID_i)`.

5 Security analysis

In this section, we first apply the CK attack model to evaluate EMAS. Through the CK model, it is usually assumed that attacker $\mathcal{A}$ has the ability to listen, modify and input the information exchanged in the communication between ES_j and ZTC_i , so as to control the communication. $\mathcal{A}$ can interact with and communicate ES_j or ZTC_i . Then we conclude EMAS has sufficient security features by theoretical analysis.

5.1 Formal evaluation by random oracle model

Through random oracle models, all participants in the model can interact with each other. The scheme is considered an ideal system if and only if any competitor with a particular capability has only a negligible probability of success.

Theorem Assume that $\mathcal{A}$ is a polynomial time adversary. If the attacker $\mathcal{A}$ forges the identity of the ZTC_i , or the ES_j , and completes the two-way authentication process, it means that the attacker $\mathcal{A}$ can initiate an impersonation attack at the both sides to break the semantic security as defined in Sect. 3.3. Assuming that $\mathcal{A}$ knows the authentication process of the proposed scheme given a certain probability δ . After $\mathcal{A}$ obtains the ID of the ES_j or ZTC_i , $\mathcal{A}$ will use this identity to join the network, thereby obtaining the authentication information to break the semantic security. The advantage of $\mathcal{A}$ breaking scheme Σ is defined as $\text{adv}_A^{\text{MA}} = P_r[ID'_j = ID_j \cup ID'_i = ID_i]$. As long as $\mathcal{A}$ breaks the two-way authentication at either side, it can be regarded as breaking the semantic security to the protocol.

$\mathcal{A}$ can issue Hash query, Send query, ESReveal query or corrupt query, which can destroy the semantic security of EMAS scheme. The advantage of $\mathcal{A}$ is defined as Eq. 3, where N is the set of integers, q_h is the number of Hash queries executed, q_s is the number of Send queries executed, and l is the size of the hash function space.

$$\text{adv}_A^{\text{MA}} \leq \frac{q_s^2}{N} + \frac{q_h^2 + q_h^3}{2l} + 2q_h \max \text{adv}_{\text{CMBDLP}}^{\text{MA}}(A), \text{adv}_{\text{CMBDHP}}^{\text{MA}}(A) \quad (3)$$

We verify the semantic security of EMAS by designing a series of attack games. From the beginning of Game GM_0 to Game GM_4 as shown Fig. 8. The authentication will continuously strengthen the capability of attacker $\mathcal{A}$, so as to verify EMAS's security.

Game GM_0 : GM_0 is to simulate the real attack of $\mathcal{A}$ under the random model. So we can get Eq. 4.

$$\text{adv}_A^{\text{MA}} = |2\text{Pr}[\text{Succ}_0] - 1| \quad (4)$$

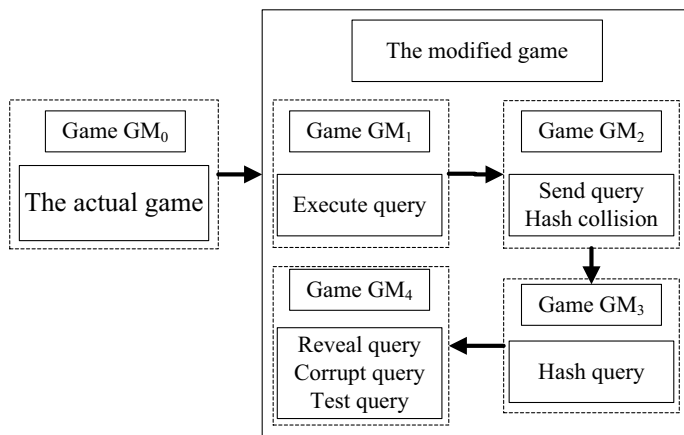


Fig. 8 The game process of CK model

Game GM_1 : Through GM_1 , $\mathcal{A}$ makes a passive attack on ZTC_i and ES_j . Attacker $\mathcal{A}$ launches Execute query to steal interaction information $\{X_i, Q_i, K_{ip}, AID_i, Y_i, Q_j, K_{jp}, ID_j\}$. but $\mathcal{A}$ cannot obtain x_i or y_j , so $\mathcal{A}$ cannot calculate the session key, which makes GM_1 no advantage over GM_0 , so we can conclude Eq. (5):

$$\Pr[\text{Succ}_1] = \Pr[\text{Succ}_0] \quad (5)$$

Game GM_2 : GM_2 is used to simulate GM_1 , and during this process, $\mathcal{A}$ makes an active attack to Send query. The following is the description of the interaction message received by $\mathcal{A}$ after sending Send query.

$\mathcal{A}$ first launches $\text{Send}(ES_j, \{X'_i, Q'_i, K'_{ip}, AID'_i\})$ query. ES_j will calculate ZTC'_i , and call Algorithm 3 to check the validity of ZTC'_i . ZTC'_i is valid only if $\mathcal{A}$ is a legitimate registered user by TA. $\mathcal{A}$ may forward authentication requests from previous ZTC_i . Then ES_j will call Algorithm 5, and check whether $Q'_i \cdot P == X'_i + (K'_{ip} + PK \cdot H_1(K'_{ip}, AID'_i)) \cdot H_3(K_{is}, AID'_i, X'_i)$ holds. If so, ES_j will authenticate $\mathcal{A}$. ES_j selects the random number $y_j \in Z_q^*$ and calculates $Y_j = y_j \cdot P$, $K_1 = y_j \cdot X'_i$ and $Q_j = y_i + k_{js} \cdot H_3(K_{js}, ID_j, Y_j)$, ES_j sets the session key $SK_{ji} = H_0(AID_i, ID_j, K_1)$ and sends $\{Y_i, Q_j, K_{jp}, ID_j\}$ to $\mathcal{A}$.

The $\text{Send}(ZTC_i, \{Y'_j, Q'_j, K'_{jp}, ID'_j\})$ query is similar to $\text{Send}(ES_j, \{X'_i, Q'_i, K'_{ip}, AID'_i\})$ query, and hence we omit the process here.

For EMAS, the random number $\{x_i, y_j\}$ is chosen from Z_q^* . If a random number collision occurs, the game will be terminated. We analyze it according to the birthday paradox. The maximum probability of collision of the Hash function is $q_h^3/2^{l+1}$. And the maximum probability of collision of the random number x_i, y_j is $q_s^2/2N$. By the difference lemma, we have Eq. 6:

$$|\Pr[\text{Succ}_2 - \text{Succ}_1]| \leq q_s^2/2N + q_h^3/2^{l+1} \quad (6)$$

Game GM_3 : In GM_3 , we assume that $\mathcal{A}$ is very lucky to guess the values of $\{Y_i, Q_j, K_{jp}\}$ without performing a Hash query. Thus, we obtain Eq. 7 as follows.

$$|\Pr[\text{Succ}_3 - \text{Succ}_2]| \leq q_s/2^{l+1} \quad (7)$$

Game GM_4 : In GM_4 , the focus is mainly on analyzing and verifying the security of the session key. Once $\mathcal{A}$ obtains the session key, the security of the key agreement protocol becomes meaningless. Compared to GM_3 , in GM_4 , $\mathcal{A}$ will have increased capabilities by launching a Reveal query and a Corrupt query in an attempt to obtain the session key. There are four possible scenarios.

Case 1: $\mathcal{A}$ launches $\text{ESReveal}(ZTC_i)$ query and $\text{ESReveal}(ES_j)$ query, by which $\mathcal{A}$ can x_i and y_j , respectively.

Case 2: $\mathcal{A}$ launches $\text{ESReveal}(ZTC_i)$ query and $\text{Corrupt}(ES_j)$ query, by which $\mathcal{A}$ can x_i and $\{PID'_j, ID_j\}$, respectively.

Case 3: $\mathcal{A}$ launches $\text{Corrupt}(ZTC_i)$ query and $\text{Corrupt}(ES_j)$ query, by which $\mathcal{A}$ can $\{PID'_i, AID_i\}$ and $\{PID'_j, ID_j\}$, respectively.

Case 4: $\mathcal{A}$ launches Corrupt (ZTC_i) query and Corrupt (ES_j) query, by which $\mathcal{A}$ can $\{PID'_i, AID_i\}$ and y_j , respectively.

After the analysis of the above four cases, it can be concluded that the session key is determined by K_1, AID_i, ID_j , and that when $\mathcal{A}$ does not solve the CMBDLP and CMBDHP assumptions, the session key cannot be obtained. As long as the assumptions of CMBDLP and CMBDHP hold, the game GM_4 is the same as the game GM_3 , as shown in Eq. 8.

$$|\Pr[Succ_4 - Succ_3]| \leq q_h \max \text{adv}_{\text{CMBDLP}}^{\text{MA}}(A), \text{adv}_{\text{CMBDHP}}^{\text{MA}}(A) \quad (8)$$

All return values from GM_0 to GM_4 are simulated by the oracle, so the Test query needs to be initiated by $\mathcal{A}$ to obtain the value of the final game b, we have Eq. 9.

$$\Pr[Succ_5] = 1/2 \quad (9)$$

Combining Eq. (4)–(8), we can conclude Eq. (9) as follows.

$$\text{adv}_A^{\text{MA}} \leq \frac{q_s^2}{N} + \frac{q_h^2 + q_h^3}{2^l} + 2q_h \max \text{adv}_{\text{CMBDLP}}^{\text{MA}}(A), \text{adv}_{\text{CMBDHP}}^{\text{MA}}(A) \quad (10)$$

6) There is a number ε , $\varepsilon > 0$ such that $\text{adv}_A^{\text{MA}} < \varepsilon$, which meets the protocol security definition of the CK adversary model. The conclusion we can draw from the above argument is that EMAS is secure enough under the random oracle model.

5.2 Informal security analysis

5.2.1 Anonymity

The anonymity protection for EMAS means that the real identity of ZTC_i is replaced with the corresponding anonymous identity AID_i . The anonymous identity $AID_i = ID_i \oplus H_1(PK, s)$ is calculated by the PDC and sent through the secure channel, and the corresponding anonymous identity will be re-issued when the system parameters are updated. Neither messages sent over the common channel nor packets provided contain real identity ID_i . Therefore, EMAS provides anonymity protection.

5.2.2 DDOS attack

A DDOS attack is a malicious cyber attack designed to disable a targeted system, network, or service by overloading it. The EMAS sets a smart contract algorithm, ES_j calculates $PID'_i = H_4(K_{ip}, AID_i)$ to call the smart contract algorithm query_EMAS(PID'_i), in order to quickly judge whether it is a request from a registered user. Similarly ZTC_i computes $PID'_j = H_4(K_{jp}, ID_j)$ to call the smart contract algorithm query_EMAS(PID'_j), which can quickly determine whether it is a response from a registered server. Therefore, EMAS can resist DDOS attacks.

5.2.3 Man-in-the-middle attack

During the authentication phase between ZTC_i and ES_j , ZTC_i calculates an authentication request message $Q_i = x_i + k_{is} \cdot H_3(K_{is}, AID_i, X_i)$, which contains the random number x_i and private key k_{is} of ZTC_i . The attacker $\mathcal{A}$ cannot obtain these key values. So $\mathcal{A}$ cannot construct valid authentication request information. Similarly, in the authentication reply message, ES_j constructs $Q_j = y_j + k_{js} \cdot H_3(K_{js}, ID_j, Y_j)$, which contains the random number y_j and private key k_{js} of ES_j . The attacker $\mathcal{A}$ cannot obtain these key values, so $\mathcal{A}$ cannot construct valid authentication reply information. Therefore, EMAS is resistant to man-in-the-middle attacks.

5.2.4 Privilege insider attack

A privilege insider attack is an attempt by an ES_j to use its privilege to fake the legal identity of ZTC_i or to allow ZTC_i to join the network without PDC 's permission. First, $\mathcal{A}$ needs to modify the data $PID_i = H_4(K_{ip}, AID_i)$ on the blockchain requiring $\mathcal{A}$ to complete an attack on 51% of the nodes on the blockchain, which is very difficult. ZTC_i attempting to join the network must pass the identity authentication of $Q_i \cdot P = X_i + (K_{ip} + PK \cdot H_1(K_{ip}, AID_i)) \cdot H_3(K_{is}, AID_i, X_i)$, in which, PK is the public key of PDC , K_{is} is the public key issued by PDC . So ZTC_i must be approved by PDC to pass the identity verification. Therefore, EMAS can resist privilege insider attacks.

6 Performance evaluation

In this section, we analyze the performance of EMAS scheme. We have selected three representative works for comparative analysis, namely AKEP [25], TPAK [9] and BAAK [11]. Among them, AKEP and TPAK are based on elliptic curve cryptography (ECC) without incorporating blockchain technology, whereas BAAK embraces the utilization of blockchain. While AKEP exhibits commendable security measures, its performance is subpar. On the other hand, AKEP demonstrates satisfactory performance but entails certain risks in terms of compromising security, such as susceptibility to KCI attacks. We evaluate performance from four aspects, namely number of cryptographic operations, computation delay, communication costs, and execution time of smart contract.

6.1 Number of cryptographic operations

In the four schemes, the cryptographic operations used are, respectively, hash operation, scalar multiplication operation, point addition operation, symmetric key encryption operation, symmetric key decryption operation, and their symbols are expressed as: Has, Mul, Add, Enc, Dec, and Che. For a more intuitive comparison of performance data between schemes. We unify the names of the communication entities in each scheme. The following is the definition of the name of the

Table 3 Execution time of basic operations (ms)

	Description	Server (ms)	User (ms)
T_{Has}	Execution time of Hash operation (SHA-256, 160bit)	0.021436	0.072978
T_{Enc}	Execution time of symmetric encryption operation (AES-128, ECB mode, 128bit)	0.012733	0.050061
T_{Dec}	Execution time of symmetric decryption operation (AES-128, ECB mode, 128bit)	0.017065	0.081415
T_{Mul}	Execution time of ECC point multiplication operation (512 bits)	4.593864	23.440564
T_{Add}	Execution time of ECC point addition operation (512 bits)	0.049233	0.165217

Table 4 Comparison of number of cryptography operations

	EMAS (Initial authentication)		EMAS (Continuous authentication)		TPAK [9]		BAAK [11]		AKEP [25]	
	User	Server	User	Server	User	Server	User	Server	User	Server
Has	3	3	2	2	6	4	5	6	8	4
Mul	2	2	0	0	3	4	4	4	4	3
Add	0	0	0	0	1	2	1	1	5	1
Enc	0	0	2	2	0	0	0	0	1	0
Dec	0	0	2	2	0	0	0	0	1	0

communication entity: (1) The user side: the device on the side of recording and collecting the user's electricity consumption information; (2) The edge server side: the device on the side of issuing and processing the user's electricity consumption information. The authentication process between the user and the server is implemented through simulation. Using two devices that can represent the user and the server, simulates the authentication process for both parties. The analog user uses the RASPBERRY PI 3B+, a chip with 1 GB LPDDR2 SDRAM and BCM2837B0 systems at a frequency of 1.4 GHz. The emulation server uses a computer with 4 GB of RAM and an INTEL(R) CELERON (R) J1900 CPU [31, 32]. The time required to perform each cryptographic operation on the two analog machines is shown in Table 3. Table 3 shows that both on the user side and server side, the Mul operation takes the longest and the Enc operation takes the shortest.

To compare the performance of schemes in terms of the number of cryptographic operations, we collect statistics on the usage of different encryption operations in each scheme, as shown in Table 4. The other three schemes use more cryptographic operands, which can increase their computation delay. And EMAS uses a relatively small number of Mul encryption operations, which also reduces computation delay.

6.2 Computation delay

The lower the computation delay of the scheme, the shorter the time for the device to process the data, and the faster the device can process more other data, improving the working efficiency of the device. Based on the information in Tables 2 and 3, the total number of cryptographic operations used by each scheme and the total time taken to execute the encryption operands are calculated. The total number of cryptographic operations used by the TPAK scheme is 20, and the execution of encryption operands takes 79.151 ms, as shown in Eq. 11. The total number of encryption operations used by the AKEP scheme is 27, and the execution of encryption operations takes 109.143 ms, as shown in Eq. 12. The BAAK scheme uses 21 encryption operations in total, and it takes 103.238 ms to execute the cryptographic operations, as shown in Eq. 13. The EMAS scheme uses a total of 10 cryptographic operations, which take 49.309 ms to execute, as shown in Eq. 14.

$$\begin{aligned} T_{\text{EMAS-Initial-Authentication}} &= 3T_{\text{Has}}^U + 3T_{\text{Has}}^S + 2T_{\text{Mul}}^S + 2T_{\text{Mul}}^U = 49.309 \text{ ms} \\ T_{\text{EMAS-Continuous-Authentication}} &= 2T_{\text{Has}}^U + 2T_{\text{Has}}^S + 2T_{\text{Dec}}^S + 2T_{\text{Dec}}^U + 2T_{\text{Enc}}^U + 2T_{\text{Enc}}^S = 0.511 \text{ ms} \end{aligned} \quad (11)$$

$$T_{\text{EMAS}} = 6T_{\text{HAS}}^S + 4T_{\text{HAS}}^S + 3T_{\text{Mul}}^U + 4T_{\text{Mul}}^S + T_{\text{Add}}^U + 2T_{\text{Add}}^S = 79.751 \text{ ms} \quad (12)$$

$$T_{\text{BAAK}} = 4T_{\text{Mul}}^U + 4T_{\text{Mul}}^S + T_{\text{Add}}^S + T_{\text{Add}}^U + 5T_{\text{Has}}^U + 6T_{\text{Has}}^S = 103.238 \text{ ms} \quad (13)$$

$$T_{\text{AKEP}} = 8T_{\text{Has}}^U + 4T_{\text{Has}}^S + 4T_{\text{Mul}}^U + 3T_{\text{Mul}}^S + 5T_{\text{Add}}^U + T_{\text{Add}}^S + T_{\text{Enc}}^U + T_{\text{Dec}}^U = 109.143 \text{ ms} \quad (14)$$

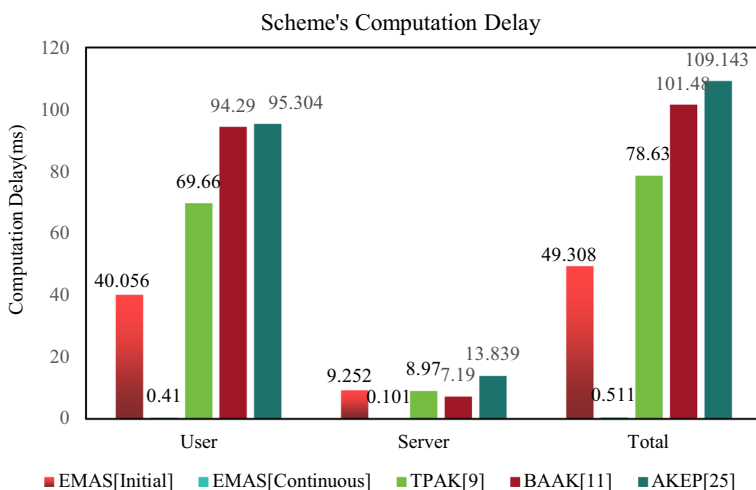


Fig. 9 Computation delay of each scheme in the authentication phase

Figure 9 shows the computation delay of the four schemes at different sides. On the user side, the AKEP scheme has the largest computation delay of 95.304 ms. The computation delay of TPAK and BAAK schemes is 69.66 ms and 94.29 ms, respectively. The EMAS scheme has the smallest computation delay of 40.057 ms, which is 58% smaller than the AKEP scheme, 42% smaller than the TPAK scheme, and 57% smaller than the BAAK scheme. On the server side, the AKEP scheme has the largest computation delay of 13.839 ms. The computation delay of TPAK and BAAK schemes is 8.97 ms and 7.19 ms, respectively. The computation delay of EMAS scheme is 9.25 ms which is 33% smaller than that of the AKEP scheme and similar to TPAK and BAAK. In summary, EMAS scheme has the lowest computing delay on the user side. It can better release the computing power of the user device, improve the computing speed of the user device, reduce the load of the user device, and reduce the pressure of the user device. On the server side, the EMAS scheme also has low computation latency. Compared with the other three schemes, EMAS scheme has lower computing delay in both user and server. Therefore, EMAS scheme has certain performance advantages in computation delay. It is worth noting that our EMAS adopts a zero-trust architecture and uses continuous validation, but compared to the initial authentication, the continuous authentication delay is very small, which consumes only 0.411ms in total.

In Sect. 4, we analyze the security features of EMAS and prove that EMAS can withstand some attacks. Currently, attacks can be divided into two categories: known attacks and unknown attacks. In this article, attacks that security analysis proves can be defended against are considered known attacks. Other potential attacks that have not been proven by security analysis are considered unknown attacks, which are difficult to predict. In order to analyze the scheme comprehensively, we also add the scenario of an unknown attack on the identity authentication process, and analyze the performance of each scheme in this scenario. It is assumed that unknown attacks will increase the authentication delay and even interrupt the authentication process. Known attacks cannot interrupt the authentication process of these four schemes. Under known attacks, the computation delay of each scheme is fixed. But under an unknown attack, the computation delay is uncertain. In order to evaluate the performance of these four schemes under different unknown attacks. We use C++ to simulate the process of these four authentication schemes under unknown attacks. In the simulation, the attack consists of two parts: known attack and unknown attack. By changing the proportion of known attacks and unknown attacks, the influence of each type of attack on the calculation delay of the scheme is analyzed. The average authentication delay is used as a parameter to measure the performance, and its definition is shown in Eq. 15. $delay_{unknown}$ represents the uncertain delay during each authentication process, $delay_{known}$ represents the fixed delay during each authentication process, and $times_{known}$ represents the number of repeated experiments. The average successful calculation delay can reduce the influence of individual extreme data on the whole experiment and improve the reliability of the data.

$$delay_{AVG} = \frac{delay_{unknown} * times_{known} + delay_{known} * times_{known}}{times_{known}} \quad (15)$$

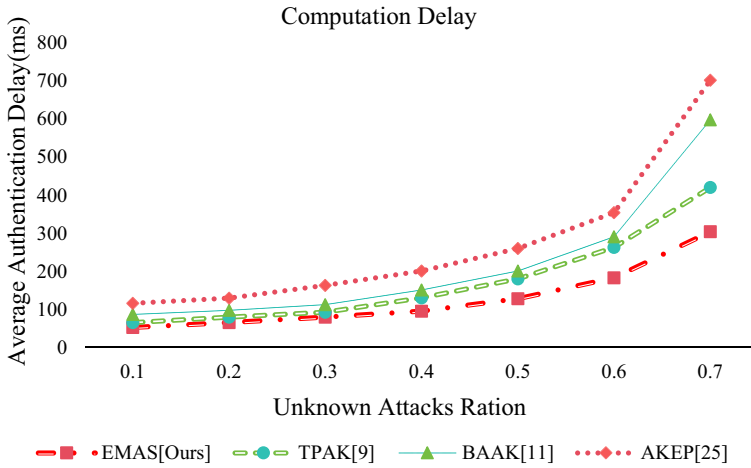


Fig.10 Computation delay of each scheme under different attack ratios

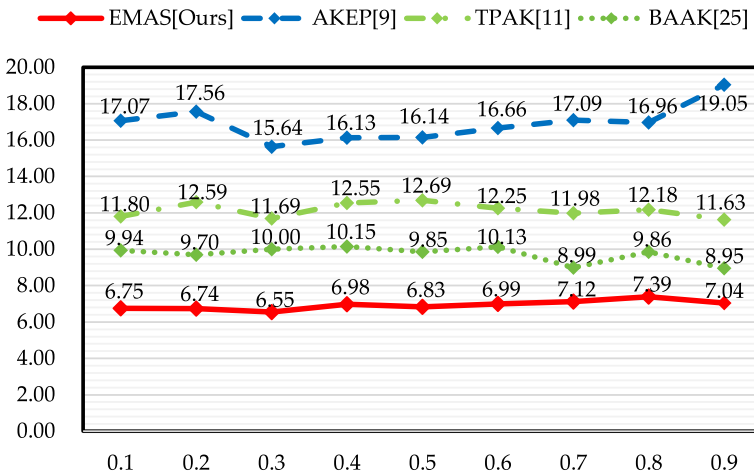


Fig. 11 Standard deviation of computation time of each scheme under different attack ratios

Figure 10 shows the average successful authentication delay of these four schemes under different attack ratios. In Fig. 10, the vertical axis represents the average successful authentication delay, and the horizontal axis represents the proportion of unknown attacks. Four curves of different colors represent four schemes. Figure 10 shows that the average successful authentication delay for each scheme increases as the proportion of unknown attacks increases. However, compared with the other three schemes, EMAS still has the lowest average successful authentication delay. This shows that even in the environment of unknown attacks, EMAS also has certain advantages in terms of computation delay.

We also simulate the standard deviation of the computation delay of each scheme under different attack proportions, as shown in Fig. 11. The standard deviation represents the stability of the scheme. If the standard deviation of the calculation delay of the scheme is larger, it indicates that the identity authentication scheme is not stable and cannot resist some unknown attacks, resulting in identity authentication failure. If the calculation delay standard deviation of the scheme is smaller, it indicates that the identity authentication scheme is stable, and there is no difference between unknown attacks of different types. As can be seen from Fig. 11, the standard deviation of EMAS is the smallest under different attack proportions, indicating that EMAS has the most stable computation delay and the least fluctuation under different attack environments.

6.3 Communication costs

The communication cost between the user and server can be dissected into two main components: transmission delay and propagation delay. Transmission delay denotes the time taken for a message to travel from the source to the destination, influenced by the data rate of the communication channel. In this scenario, WLAN serves as the communication channel between the user and server, with a data rate of 100 Mbps for both uplink and downlink communication. On the other hand, propagation delay refers to the time it takes for a message to physically traverse from the source to the destination, influenced by the distance between the parties and the electromagnetic wave's speed. In wireless communication, the wave propagation speed is assumed to be 3×10^8 m/s. Given a communication distance of 150 m between the user and server, with a message propagation speed of 3×10^8 m/s, the propagation delay between the user and server amounts to 0.5 μ s.

Additionally, the communication overhead for a hash operation is 160 bits. A random number, identity identifier (ID), and timestamp T each necessitate 32 bits. Each operation for a point on the elliptic curve uses 161 bits. An element in the multiplication group requires 512 bits, while RSA encryption/decryption operations utilize 1024 bits.

The communication overhead of each scheme is shown in Table 5, among which the largest communication overhead of the AKEP scheme is 1952 bits. The communication overhead of TPAK scheme is 1408 bits, and the communication overhead of BAAK scheme is 1568 bits. The communication overhead of EMAS

Table 5 Communication cost of each scheme in authentication

Scheme	Communication overhead (bit)	Transmission delay (μ s)	Propagation delay (μ s)	Total (μ s)
TPAK [23]	1408	1408	1	15.08
AKEP [12]	1952	19.52	2	21.52
BAAK [15]	1568	15.68	1	16.68
EMAS	1536	15.36	1	16.36

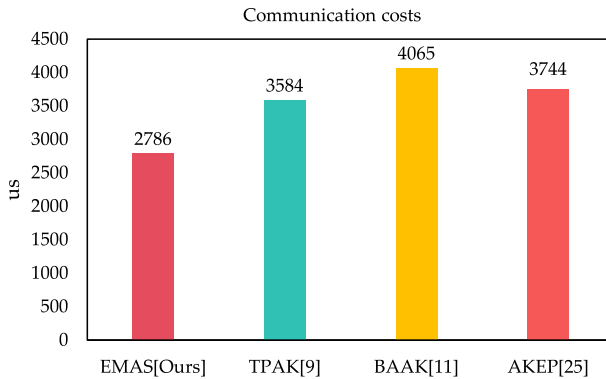


Fig. 12 Communication costs of each scheme in authentication

is 1536 bits. The communication overhead consumed by EMAS remains consistent in the comparison scheme. EMAS effectively addresses security vulnerabilities, ensures minimal communication overhead, and fulfills the communication requirements of the power IoT.

As shown in Table 5 and Fig. 12, the total communication cost of the TPAK scheme is 15.08 μ s, the SCDC scheme is 23.76 μ s, the SEMA scheme is 22.12 μ s, the AKEP scheme is 21.52 μ s, the BAAK scheme is 16.68 μ s, and the XMAS scheme is 10.24 μ s. XMAS has reduced communication costs by 56.90% compared to the SCDC scheme, 38.61% compared to the BAAK scheme, 53.71% compared to the SEMA scheme, 52.42% compared to the AKEP scheme, and 32.12% compared to the TPAK scheme. So, the proposed XMAS has been shown to hold the lowest communication cost.

6.4 Execution time of smart contract

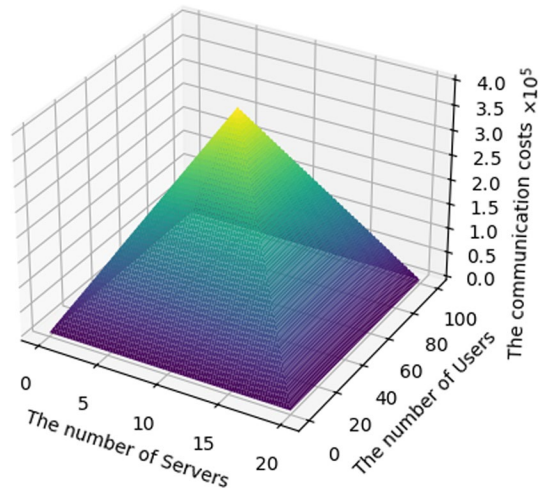
In order to evaluate the performance of smart contracts, we built a FISCO BCOS alliance chain. The blockchain runs on the x86_64Linux system, Ubuntu version 18.04.6, and generates a single group of 4-node FISCO chain with the Ethereum Virtual Machine (EVM) engine, where the consensus mechanism adopts PBFT algorithm. When conducting a smart contract deployment operation, such as an update_EMAS operation, the initial step is to determine the start time as the commencement point of execution. The process of smart contract deployment typically

Table 6 Execution time cost (s) of the smart contract

	initialization_ EMAS	update_EMAS	query_EMAS	revoke_EMAS
Max Time	0.1881	0.2732	0.0546	0.3375
Min Time	0.1532	0.1957	0.0228	0.2015
Ave Time	0.1687	0.2315	0.0342	0.2414

Table 7 Time cost of the scheme

Scheme	Cost of smart contract (ms)	Cost of computation (ms)	Communication cost (us)	Total
EMAS	468.6	49.309	16.36	517.909 ms + 16.36 us
BAAK	700.1	106.143	16.68	806.243 ms + 16.68 us

Fig. 13 Communication costs for CRL-based revocation (B)

commences with initiating a deployment transaction. This stage involves utilizing the relevant interfaces or tools provided by the FISCO blockchain to submit transaction requests to the network for smart contract deployment. Subsequently, confirmation of the transaction within the blockchain network becomes necessary. The duration of this waiting process is influenced by various factors such as network load, transaction complexity, and communication delays between nodes. Once the transaction is confirmed and successfully recorded on the blockchain, this moment in time is documented as the endpoint denoting the completion of smart contract deployment. The time required for smart contract deployment can be calculated as the difference between these two points in time (end time minus start time). By repeating this procedure 100 times, data regarding maximum, minimum, and average execution times can be obtained to evaluate efficiency and performance levels associated with smart contract deployments. The running time of the algorithms is shown in Table 6.

We define the sum of the smart contract runtime, the authentication time and the communication cost as the total which is measured by the average time in Table 7. Total is used to evaluate the performance of EMAS scheme and BAAK scheme, as shown in Table 7. The EMAS scheme invokes “initialization” once, “update” once, “query” twice, and “authenticate” once, whereas the BAAK scheme invokes “initialization” once, “update” twice, “query” twice, and “authenticate” once. Consequently, the total smart contract time for the EMAS scheme is 710 ms and for the BAAK scheme is 941.5 ms. The computation time is 49.309 ms and 106.143 ms for

Fig. 14 Communication costs for BC-based revocation (B)

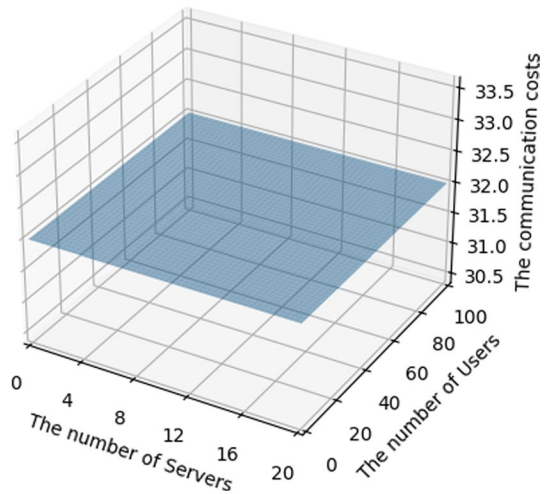
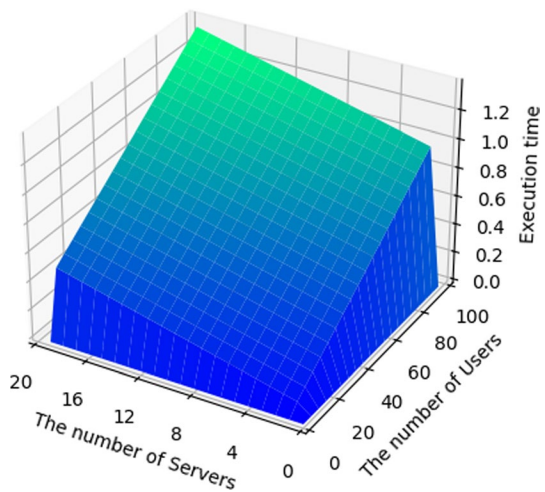


Fig. 15 The delays of the execution of smart contract (s)



EMAS and PAAK, respectively, as shown in Eq. (11) and Eq. (13). So the total time of EMAS is 759.309 ms and 16.36 μ s, and BAAK is 1047.643 ms and 16.68 μ s. EMAS is 35.8% lower than BAAK.

When it comes to revocation, the authentication delay at the edge server's side may reach up to 248.5 ms due to the 241.4 ms cost of revocation_{EMAS}. Although this may appear slightly higher, it remains practical for smart grid systems as a real edge server should possess significantly more computational resources than our simulation platform. Furthermore, there are no additional computation costs on the user's side. Figure 13 illustrates the communication costs of revocation utilizing the CRL. It is evident that these costs will escalate rapidly with an increase in both revoked users and edge servers' numbers. However, it is apparent that in our proposed EMAS, based on blockchain technology, the communication costs of

revocation remain constant (i.e., equal to the length of PID). This is because it only requires invoking the smart contract `revoke_EMAS(PID)` by providing a parameter PID as depicted in Fig. 14. Therefore, considering both security and efficiency aspects, our proposed EMAS proves more suitable for edge computing-based smart grid systems.

To assess the performance of scalability of our proposed EMAS, we conduct simulations by progressively increasing the number of servers and users, as depicted in Fig. 15. The simulation results demonstrate that with an expansion in user count, the system's execution time also increases accordingly. This growth phenomenon can be primarily attributed to the following factors: First, as the number of users rises, the system encounters a greater volume of authentication requests, leading to an increase in execution time. Second, as more servers are added, each server necessitates data synchronization with other peers. This synchronization process contributes to a longer execution time. Notably, augmenting the number of server nodes has a more pronounced impact on system performance compared to increasing user count due to heightened complexity and network communication overhead associated with data synchronization operations on servers. Consequently, we conclude that while enlarging both server and user numbers leads to increased execution time, overall execution remains within acceptable levels.

6.5 Discussion on results

We propose the EMAS mutual authentication scheme based on ECC, which incorporates blockchain technology and a zero-trust architecture. The zero-trust engine is deployed at the edge of the power IoT, in close proximity to PTD, effectively mitigating single points of failure and preventing denial-of-service attacks. EMAS eliminates the need for administratively burdensome certificate operations and optimizes blockchain content storage. Additionally, during the authentication process, we strive to minimize time-consuming ECC point multiplication operations while ensuring security. In terms of smart contract deployment on the blockchain, only PID information is required for revocation or update operations. Interactions with the blockchain during authentication are streamlined by completing most tasks during registration. Our EMAS maintains good performance without sacrificing safety.

7 Conclusion

We propose an ECC-based mutual authentication scheme for the cloud-edge-end collaboration power IoT environment based on blockchain technology to realize secure bidirectional authentication between power terminal edge server, applying blockchain technology to avoid certificate management and the problem of a single point of failure. Through the comprehensive analysis and test of CK model, the proposed EMAS scheme proves that it can resist various attacks to ensure the security of the power IoT environment. By analyzing the data obtained from the simulation experiment, it can be concluded that EMAS has advantages in efficiency and

communication cost and has certain stability. EMAS is still not the lowest communication cost scheme in the comparison, so we need to further study how to reduce the communication cost, which is a topic for future research. With the continuous progress of information technology and cryptography, the power IoT identity authentication technology will also continue to innovate and optimize to adapt to the more complex and changeable power system environment and security needs.

Acknowledgements This research is supported by the project "Research on edge adaptive security interaction technology of power IoT" (No. 546826230037) by Beijing FibrLink Communications Co. Ltd, and also funded by the project "Low cost distribution network tower tilt and outer break early warning system based on MESH ad-networking technology" (No. XX92000Y07) by the Automation Technology Project of State Grid Zhejiang Xinxing Technology Co., LTD.

Author contributions L. and W. wrote the main manuscript text and Z.S. and M. prepared figures 1-10. All authors reviewed the manuscript.

Declarations

Conflict of interest The authors declare no competing interests.

References

1. Sawas AU, Farag HEZ (2023) Real-time detection of stealthy IoT-based cyber-attacks on power distribution systems: a novel anomaly prediction approach. *Electr Power Syst Res*. <https://doi.org/10.1016/j.epr.2023.109496>
2. Envelope M, Aah A, Zs A et al (2023) Review on cyber-physical and cyber-security system in smart grid: Standards, protocols, constraints, and recommendations. *J Netw Comput Appl*. <https://doi.org/10.1016/j.jnca.2022.103540>
3. Huang W, Xie X, Wang Z et al (2023) ZT-Access: a combining zero trust access control with attribute-based encryption scheme against compromised devices in power IoT environments. *Ad hoc networks* 145:103161
4. Uludag S, Lui KS, Ren W, Nahrstedt K (2016) Secure and scalable data collection with time minimization in the smart grid. *IEEE Trans Smart Grid* 7(1):43–54
5. Philips A, Jayaraj J, Josh F, Venkateshkumar P (2021) Enhanced RSA key encryption application for metering data in smart grid. *Int J Perv Comput Commun* 17(5):596–610
6. Srinivas MJ, Das AK, Li X, Khan MK, Jo M (2021) Designing anonymous signature-based authenticated key exchange scheme for IoT-enabled smart grid systems. *IEEE Trans Ind Inform* 17(7):4425–4436
7. Xiang XY, Cao J (2022) An efficient authenticated key agreement scheme supporting privacy-preservation for smart grid communication. *Electric Power Systems Res* 203:107630
8. Chaudhry SA, Yahya K, Garg S, Kaddoum G, Hassan M, Zikria YB (2022) Las-sg: An elliptic curve based lightweight authentication scheme for smart grid environments. *IEEE Trans Ind Inform* 19(2):1504–1511
9. Qi M, Chen J (2021) Two-pass privacy preserving authenticated key agreement scheme for smart grid. *IEEE Syst J* 15(3):3201–3207
10. Deng LZ, Wang T, Feng S, Qu YY, Li SW (2023) Secure identity-based designated verifier anonymous aggregate signature scheme suitable for smart grids. *IEEE IoT J* 10(1):57–65
11. Wang J, Wu L, Choo K-KR, He D (2020) Blockchain-based anonymous authentication with key management for smart grid edge computing infrastructure. *IEEE Trans Ind Inform* 16(3):1984–1992
12. Oumair N, Ullah S, Anjum L (2021) Blockchain-based decentralized lightweight control access scheme for smart grids. *Arab J Sci Eng* 46(9):8233–8243
13. Weizheng W et al (2021) Secure and efficient mutual authentication protocol for smart grid under blockchain. *Peer-to-Peer Netw Appl* 14(5):2681–2693

14. Bao H et al (2021) BBNP: A Blockchain-based Novel Paradigm for Fair and Secure Smart Grid Communications. *IEEE IoT J* 9(5):12984–12996
15. Qu YY, Uddin MP, Gan CQ et al (2023) Blockchain-enabled federated learning: a survey. *ACM Comput Surv* 55(4):1–35
16. Ul Hassan M, Rehmani MH, Chen JJ (2023) Anomaly detection in blockchain networks: a comprehensive survey. *IEEE Commun Surv Tutor* 25(1):289–318
17. Ran C, Krawczyk H (2001) Analysis of key-exchange protocols and their use for building secure channels. In: *International Conference on the Theory & Application of Cryptographic Techniques: Advances in Cryptology*. Springer, Berlin and Heidelberg
18. Ward R, Beyer B (2014) Beyond corp: a new approach to enterprise security. *Login Magaz USE-NIX SAGE* 39(6):61
19. Saleem M, Warsi MR, Islam S (2023) Secure information processing for multimedia forensics using zero-trust security model for large scale data analytics in SaaS cloud computing environment. *J Inform Secur Appl* 72:2214–2126
20. Chen L, Dai Z, Chen M, et al.(2021) Research on the security protection framework of power mobile Internet services based on zero trust. In: *Proceeding of 2021 16th International Conference on Smart Grid and Electrical Automation (ICSGEA)*. Piscataway, NJ: IEEE, pp 65–68
21. Sultana M, Hossain A, Laila F et al (2020) Towards developing a secure medical image sharing system based on zero trust principles and blockchain technology. *BMC Med Inform Decis Mak* 20(1):1–10
22. Feng J, Tingting Yu, Wang Z, Zhang W, Han G, Huang W (2022) AnEdgeZero trust model against compromised terminals threats in power IoT environments. *J Comput Res Develop* 59(5):1–13
23. Chang Y, Li JL, Li WJ (2023) 2D2PS: A demand-driven privacy-preserving scheme for anonymous data sharing in smart grids. *J Inform Secur Appl* 74:103466
24. Hu SF, Chen YR, Zhen YL et al (2023) Provably secure ECC-based authentication and key agreement scheme for advanced metering infrastructure in the smart grid. *IEEE Trans Ind Inform* 19(4):5985–5994
25. Abbasinezhad-Mood D, Ostad-Sharif A, Nikooghadam M (2020) Novel anonymous key establishment protocol for isolated smart meters. *IEEE Trans Industr Electron* 67(4):2844–2851
26. Javed S et al (2022) An efficient authentication scheme using blockchain as a certificate authority for the internet of drones. *Drones*. <https://doi.org/10.3390/drones6100264>
27. Abdelmaboud A et al (2022) Blockchain for IoT applications: taxonomy platforms recent advances challenges and future research directions. *Electronics*. <https://doi.org/10.3390/electronics11040630>
28. Kumar P, Lin Y, Bai G, Pavard A, Dong JS, Martin A (2019) Smart grid metering networks: a survey on security, privacy and open research issues. *IEEE Commun Surv Tutor* 21(3):2886–2927
29. Mansour DEA, Numair M, Zalhaf AS et al (2023) Applications of IoT and digital twin in electrical power systems: a comprehensive survey. *IET Generat Trans Distribut* 17(20):4457–4479
30. Park K, Lee JY, Das AK et al (2023) BPPS: blockchain-enabled privacy-preserving scheme for demand-response management in smart grid environments. *IEEE Trans Depend Secure Comput* 20(2):1719–1729
31. "Arduinolibs: Cryptographic library," URL <http://rweather.github.io/arduinolibs/crypto.html>
32. "OpenSSL, cryptography and SSL/TLS toolkit," URL <http://www.openssl.org>, 2017

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.